UNIVERSITÄT PASSAU

Machine Learning for Cybersecurity (5883V/Ü)

2. Simple Classification: Basics, Spam Messages

Hans P. Reiser | Winter term 2026/27

University of Passau



"Spam wall" by freezelight is licensed under CC BY-SA 2.0.

Overview

1. Python + ML basics
   - Data preparation – pandas
   - Training a model – scikit-learn
   - Visualization – pyplot, seaborn
2. A bit of theory: Naïve Bayes Classification
3. Homework exercise: Spam classification
   - Programming exercise
   - Reading assignment

ML+Python: Data preparation

- We'll make use of pandas

```
>>> import pandas as pd
>>> df = pd.read_csv('./creditcard.csv')

# Take a look at the data
>>> print(df.head())
   Time        V1        V2    ...         V27       V28  Amount  Class
0   0.0 -1.359807 -0.072781    ...    0.133558 -0.021053  149.62      0

>>> df.columns
Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
       'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',
       'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',
       'Class'],
      dtype='object')

>>> print(df.isnull().sum().max()) # Check for null entries
```

Data source: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?resource=download

ML+Python: Data preparation

- Querying information about the data

```
>>>  df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
...
>>> df[df.Class == 1].Amount.describe()     # filter by Class==1 (is fraud)
count     492.000000                        # take the amount column
mean      122.211321                        # describe => show statistics
std       256.683288
min         0.000000
25%         1.000000
50%         9.250000
75%       105.890000
max      2125.870000
Name: Amount, dtype: float64
```

ML+Python: Data preparation

- We'll be using scikit-learn

```
# Splitting data into features and classes
>>> x = df.drop("Class", axis=1)
>>> y = df.Class

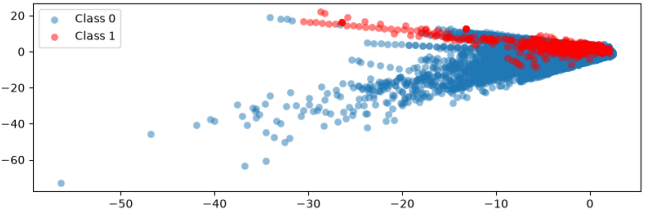
# Splitting data into training set and testing set
>>> from sklearn.model_selection import train_test_split
>>> x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
    random_state=42)
```

- Training data will be used for training a model
- Test data will be used for testing (never for training) a model

ML+Python: Visualizing data

- Sometimes it might be useful to visualize data sets

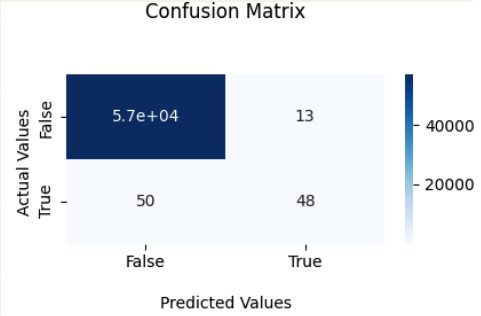
```
>>> import matplotlib.pyplot as plt
>>> xv = x.values
>>> yv = y.values
# Plot two features [1],[2] vs. honest/fraudulent transactions
>>> plt.scatter(xv[yv==0, 1], xv[yv==0, 2], label="Class 0", alpha=0.5,
    linewidth=0.2)
>>> plt.scatter(xv[yv==1, 1], xv[yv==1, 2], label="Class 1", alpha=0.5,
    linewidth=0.2, c='r')
>>> plt.legend()
>>> plt.show()
```



ML+Python: Training a model

- Example: Logistic regression for credit card fraud detection

```
1  import seaborn as sb
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.metrics import confusion_matrix
4
5  # Train model with training data
6  model = LogisticRegression(max_iter=1000)
7  model.fit(x_train, y_train)
8
9  # Test the model
10 y_test_pred = model.predict(x_test)
11 m = confusion_matrix(y_test, y_test_pred)
12 ax = sb.heatmap(m, annot=True, cmap='Blues')
13 ax.set_title('Confusion Matrix\n\n');
14 ax.set_xlabel('\nPredicted Values')
15 ax.set_ylabel('Actual Values ');
16 ax.xaxis.set_ticklabels(['False', 'True'])
17 ax.yaxis.set_ticklabels(['False', 'True'])
18 plt.show()
```

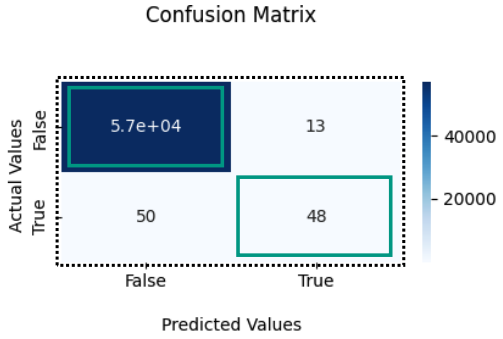


ML+Python: Training a model

- More metrics
  - Accuracy: fraction of all predictions that are correct
  - Precision: of the predicted positives, how many are actually positive
  - Recall: of the actual positives, how many we found
  - F1-score: harmonic mean of precision and recall
- More details on those metrics next week

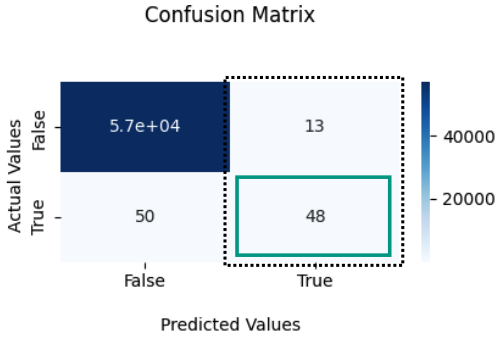
ML+Python: Training a model

- More metrics
  - **Accuracy**: fraction of all predictions that are correct
  - Precision: of the predicted positives, how many are actually positive
  - Recall: of the actual positives, how many we found
  - F1-score: harmonic mean of precision and recall
- More details on those metrics next week



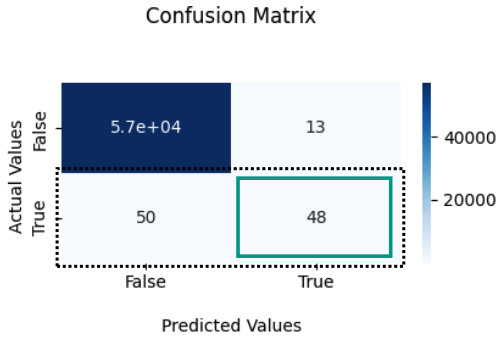
ML+Python: Training a model

- More metrics
  - Accuracy: fraction of all predictions that are correct
  - **Precision**: of the predicted positives, how many are actually positive
  - Recall: of the actual positives, how many we found
  - F1-score: harmonic mean of precision and recall
- More details on those metrics next week



ML+Python: Training a model

- More metrics
  - Accuracy: fraction of all predictions that are correct
  - Precision: of the predicted positives, how many are actually positive
  - **Recall**: of the actual positives, how many we found
  - F1-score: harmonic mean of precision and recall
- More details on those metrics next week



ML+Python: Training a model

- More metrics

```
>>> precision = precision_score(y_test, y_test_pred)
>>> accuracy = accuracy_score(y_test, y_test_pred)
>>> recall = recall_score(y_test, y_test_pred)
>>> f1score = f1_score(y_test, y_test_pred)

>>> print(f"Accuracy = {accuracy}")
Accuracy = 0.9988939995084443
>>> print(f"Precision = {precision}")
Precision = 0.7868852459016393
>>> print(f"Recall = {recall}")
Recall = 0.4897959183673469
>>> print(f"F1 Score = {f1score}")
F1 Score = 0.6037735849056604
```

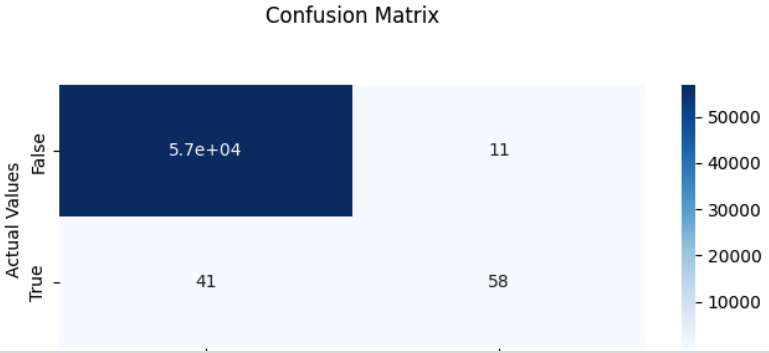
- or simply

```
>>> print(classification_report(y_test, y_test_pred))
```

ML+Python: Better preprocessing of the data set

- Amounts vary substantially in magnitude (see above). Scaling can be beneficial.

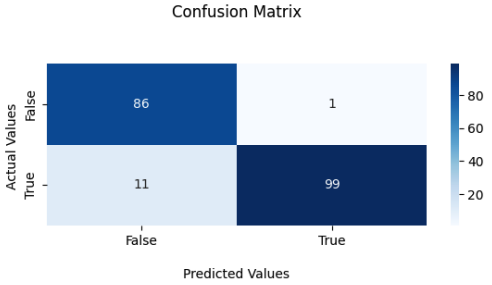
```
from sklearn.preprocessing import StandardScaler
s = StandardScaler()
x_train = x_train.copy()
x_test = x_test.copy()
x_train["Amount"] = s.fit_transform(x_train[["Amount"]])
x_test["Amount"] = s.transform(x_test[["Amount"]])
```



ML+Python: Better preprocessing of the data set

- Dataset (Class 0/1) is highly unbalanced, let's do random sub-sampling

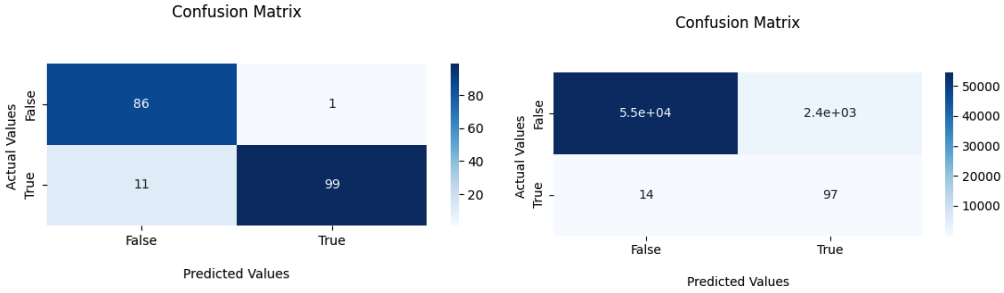
```
df = df.sample(frac=1)
num_fraud = len(df[df.Class == 1])
fraud_entries = df[df.Class == 1]
non_fraud_entries = df[df['Class'] == 0][:num_fraud]
sub_df = pd.concat([fraud_entries, non_fraud_entries])
random_undersampled_df = sub_df.sample(frac=1, random_state=42)
```



ML+Python: Better preprocessing of the data set

- Dataset (Class 0/1) is highly unbalanced, let's do random sub-sampling

```
df = df.sample(frac=1)
num_fraud = len(df[df.Class == 1])
fraud_entries = df[df.Class == 1]
non_fraud_entries = df[df['Class'] == 0][:num_fraud]
sub_df = pd.concat([fraud_entries, non_fraud_entries])
random_undersampled_df = sub_df.sample(frac=1, random_state=42)
```



Overview

1. Python + ML basics
   - Data preparation – pandas
   - Training a model – scikit-learn
   - Visualization – pyplot, seaborn
2. A bit of theory: Naïve Bayes Classification
3. Homework exercise: Spam classification
   - Programming exercise
   - Reading assignment

Basic text classification problem

Input:

- Document d
- Set of n classes $C = \{c_1, c_2, \ldots, c_n\}$

Output:

- a predicted class $c \in C$

Very basic approach: manually-coded rules (expensive!) $\Rightarrow$ Let's look at ML approaches

Basic text classification problem

Input:

- Document d
- Set of n classes $C = \{c_1, c_2, \ldots, c_n\}$
- Set of k labelled documents $(d_1, c_1), (d_2, c_2), \ldots, (d_k, c_k)$

Output:

- a trained classifier $y : d \rightarrow c$

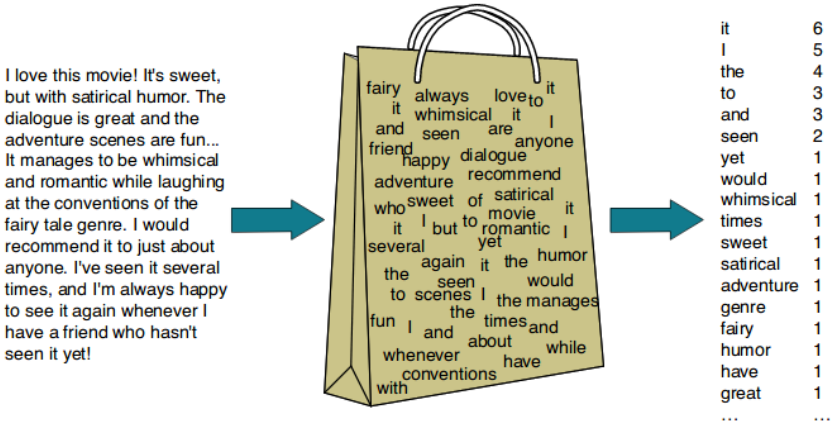
Several potentially suitable methods

- Naïve Bayes
- Logistic regression
- Support vector machines
- k-nearest neighbors

Text classification with Naïve Bayes approach

- Simple (“naïve”) classification method based on Bayes’ rule
- Uses a very simple representation of document:
  - “Bag of words”: set of words in a document (ignore order) and their frequencies

Bag of words



(image source: [Jurafsky and Martin, 2026], Chapter 11, Fig. 11.2)

Spam – Background

Background from math (statistics): Bayes' rule:

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

$$\text{posterior probability} = \frac{\text{(predictor likelihood) * (class prior probability)}}{\text{predictor prior probability}}$$

Formalizing the Bayes Classifier

- $c_{MAP} = \underset{c \in C}{\text{argmax}} P(c|d)$ (MAP: maximum a posteriori)
- $c_{MAP} = \underset{c \in C}{\text{argmax}} \frac{P(d|c)P(c)}{P(d)}$ (Bayes' rule)
- $c_{MAP} = \underset{c \in C}{\text{argmax}} P(d|c)P(c)$ (Simplified – P(d) is constant)

With document d represented as a vector of features $(x_1, x_2, \ldots, x_n)$:

- $c_{MAP} = \underset{c \in C}{\text{argmax}} P(x_1, x_2, \ldots, x_n|c)P(c)$

Can we simply estimate $P(x_1, x_2, \ldots, x_n|c)$ by observing frequencies in a training dataset?

Formalizing the Bayes Classifier

- $c_{MAP} = \underset{c \in C}{\text{argmax}} P(c|d)$ (MAP: maximum a posteriori)
- $c_{MAP} = \underset{c \in C}{\text{argmax}} \frac{P(d|c)P(c)}{P(d)}$ (Bayes' rule)
- $c_{MAP} = \underset{c \in C}{\text{argmax}} P(d|c)P(c)$ (Simplified – P(d) is constant)

With document d represented as a vector of features $(x_1, x_2, \ldots, x_n)$:

- $c_{MAP} = \underset{c \in C}{\text{argmax}} P(x_1, x_2, \ldots, x_n|c)P(c)$

Can we simply estimate $P(x_1, x_2, \ldots, x_n|c)$ by observing frequencies in a training dataset?

Problem: Huge space, would require extremely huge training set!

Naïve Bayes Classifier

- Assume **conditional independence**: Assume that feature probabilities $P(x_i|c)$ are independent given the class c
- $P(x_1, x_2, \ldots, x_n|c) = P(x_1|c)P(x_2|c)\ldots P(x_n|c)$
- We estimate $P(x_i|c)$ based on training data

Naïve Bayes Classifier

- Assume **conditional independence**: Assume that feature probabilities $P(x_i|c)$ are independent given the class c
- $P(x_1, x_2, \ldots, x_n|c) = P(x_1|c)P(x_2|c)\ldots P(x_n|c)$
- We estimate $P(x_i|c)$ based on training data

All good?

Naïve Bayes Classifier

- Assume **conditional independence**: Assume that feature probabilities $P(x_i|c)$ are independent given the class c
- $P(x_1, x_2, \ldots, x_n|c) = P(x_1|c)P(x_2|c)\ldots P(x_n|c)$
- We estimate $P(x_i|c)$ based on training data

All good?

What if $P(x_i|c) = 0$ because we did not see x_i in the training data set...

Laplace smoothing

- Solution to that problem: Pretend (in training set) that:
  - We saw one extra spam email **with** word x_i
  - We saw one extra spam email **without** word x_i

- $P(x_i|\text{spam}) = \frac{|\text{total spam emails containing } x_i|+1}{|\text{total spam emails}|+2}$
- Same for not spam

- Other implementation detail:
  - Addition of log probabilities instead of multiplication of many small probabilities to prevent floating-point underflow

Overview

1. Python + ML basics
   - Data preparation – pandas
   - Training a model – scikit-learn
   - Visualization – pyplot, seaborn
2. A bit of theory: Naïve Bayes Classification
3. Homework exercise: Spam classification
   - Programming exercise
   - Reading assignment

Programming exercise

Implement (using Python/scikit-learn) a classifier for spam email

- Dataset:
  `https://www2.aueb.gr/users/ion/data/lingspam_public.tar.gz`
- Or use some pre-processed subset:
  `https://www.dropbox.com/s/yjiplngoa430rid/ling-spam.zip`
- Preprocessing done: removal of stop-words, lemmatization (drop endings)

Tasks

- Create a dictionary of words (dropping all non-words like punctuation characters, single characters), choosing the 2000 most frequent words from training set
- Transform each mail into a word count vector
- Train a Bayes classifier (MultinomialNB from sklearn.naive_bayes)
- Check (with test set) the classifier

Research paper: Spam classification

Please read the article “Evaluating the Effectiveness of Machine Learning Methods for Spam Detection” by Kontsewaya et al. [Kontsewaya et al., 2021]

- Can you describe (at least) one alternative to naïve Bayes classification for spam detection?
- What are additional methods for dataset pre-processing that we did not discuss in the lecture?
- What are the main differences between the credit-card fraud dataset from this lecture and the spam dataset used in the paper?
- What is the “ROC curve” used in the paper? How is it calculated?
- What are, in your opinion, the most interesting aspects of this paper?
- What are, in your opinion, weaknesses of this paper?

References I

Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 3 edition, 2026. URL https://web.stanford.edu/~jurafsky/slp3/. Draft.

Yuliya Kontsewaya, Evgeniy Antonov, and Alexey A. Artamonov. Evaluating the effectiveness of machine learning methods for spam detection. In Alexei V. Samsonovich and Valentin V. Klimov, editors, *Proc. of the 2020 Annual Int. Conf. on Brain-Inspired Cognitive Architectures for Artificial Intelligence, BICA 2020*, pages 479–486. Elsevier, 2021. 10.1016/j.procs.2021.06.056.